

Plagiarism Checker and Document Analyzer Using GUI

Project submitted to the
SRM University – AP, Andhra Pradesh
for the partial fulfillment of the requirements to award the degree of
Bachelor of Technology

In
Computer Science and Engineering
School of Engineering and Sciences

Submitted by
T.Jyothi - AP23110011426
K.Teja sree - AP23110011434
G.Mohana Krishna - AP23110011462
R.Lohitha - AP23110011469



Under the Guidance of
Siva Lakshmaiah
Lecturer, Department of CSE

SRM University-AP
Neerukonda, Mangalagiri,
Guntur
Andhra Pradesh – 522 240

[April, 2025]

Certificate

Date: 10/04/2025

This is to certify that the work present in this Project entitled “**Plagiarism Checker and Document Analyzer Using GUI**” has been carried out by **OUR TEAM** under my/our supervision. The work is genuine, original, and suitable for submission to the SRM University – AP for the award of Bachelor of Technology/Master of Technology in **School of Engineering and Sciences**.

Supervisor

Siva Lakshmaiah

Designation,

Affiliation.

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

I am extremely grateful and express my profound gratitude and indebtedness to my project guide, **Siva Lakshmaiah** Department of Computer Science & Engineering, SRM University, Andhra pradesh, for her kind help and for giving me the necessary guidance and valuable suggestions in completing this project work.

T.Jyothi - AP23110011426

K.Teja sree - AP23110011434

G.MohanaKrishna - AP23110011462

R.Lohitha - AP23110011469

1. Table of Contents

Certificate	2
Acknowledgement	3
Table of Contents	4
Abstract	5
1. Introduction	6
1.1 Background	
1.1.1 Objective	
2. Methodology	7
2.1 Software Requirements	
2.2 Data Collection and Loading	
2.3 Preprocessing	
2.4 Feature Extraction	
2.5 Similarity Computation	
2.6 Sentiment Analysis	
2.7 Visualization and Reporting	
2.8 Plagiarism Detection	
2.9 Export Feature	
2.10 Justification of Methodology	
3. Implementation	9
4. Result and Analysis	10
5. Discussion and Conclusion	14
6. Future Work	14
7. References	15

Abstract

This project presents an intelligent, interactive, and user-friendly GUI-based application designed to detect plagiarism among text documents. With the rise of academic and digital content, there is a growing need for automated tools that can quickly analyze and compare large volumes of documents for similarities. This tool addresses that need by allowing users to load multiple .txt files and perform deep text analysis using machine learning and natural language processing techniques.

The core functionality is built using Python and leverages libraries such as NLTK for text processing and sentiment analysis, Scikit-learn for TF-IDF vectorization and cosine similarity, Matplotlib for data visualization, and WordCloud for word frequency visualizations. The application features a graphical user interface created using Tkinter, making it accessible to users without any programming knowledge.

Key features include document similarity detection, real-time plagiarism alerts, word frequency visualization through word clouds and bar charts, sentiment analysis, and the ability to export similarity results into a CSV file. A threshold-based plagiarism checker visually highlights suspiciously similar document pairs, ensuring transparency and ease of review.

This tool provides a practical solution for institutions and individuals seeking to maintain originality and academic integrity in written content. Its modular design, ease of use, and extensibility position it as a strong candidate for further development into a comprehensive, multi-format plagiarism detection platform.

1. Introduction

Plagiarism not only undermines ethical standards but also hampers creativity and fair academic assessment. Manual plagiarism detection is not only inefficient but also prone to error, especially when dealing with large volumes of data or subtly paraphrased content. Hence, automated systems that can evaluate the similarity between documents and provide objective, quantifiable analysis have become essential in modern educational and publishing settings.

This project introduces a GUI-based plagiarism checker application that uses Natural Language Processing (NLP) and Machine Learning (ML) techniques to detect similarities between text documents. The tool enables users to load a collection of text files and compare them for duplication using TF-IDF (Term Frequency-Inverse Document Frequency) and cosine similarity metrics. The results are visualized using heatmaps and similarity matrices, providing an intuitive understanding of textual overlap.

1.1 Background

The exponential growth of digital content has made plagiarism a widespread issue. Traditional plagiarism detection methods involve manual reading, comparison, or keyword matching, which can be time-consuming and unreliable for large datasets. With increasing academic and publishing demands, there is a significant need for automated systems that can handle these tasks efficiently.

Advancements in Natural Language Processing have made it possible to computationally analyze language and determine patterns, similarities, and sentiment. By incorporating these tools into a user-friendly application, this project bridges the gap between complex NLP techniques and their real-world applications.

1.1.1 Objective

The primary objective of this project is to develop a GUI-based system for efficient and accessible document plagiarism detection. The system aims to:

- Automate the loading and processing of multiple text files
- Calculate document similarity using TF-IDF and cosine similarity
- Provide visual feedback through similarity matrices, word clouds, and bar graphs
- Analyze sentiment and tone within each document
- Detect plagiarism based on a similarity threshold and notify the user
- Allow users to export analysis results to CSV format

2. Methodology

The methodology for the development of the Plagiarism Checker Using GUI is structured into several distinct phases, covering software setup, data handling, processing, analysis, and visualization. This section outlines the systematic approach used to create an intuitive, functional, and effective document analysis tool.

2.1 Software Requirements

- **Programming Language:** Python 3.10 or later
- **GUI Framework:** Tkinter (for building the graphical interface)
- **Libraries Used:**
 - nltk: for text preprocessing and sentiment analysis
 - sklearn: for TF-IDF vectorization and cosine similarity
 - matplotlib: for plotting visual data (heatmaps, bar charts)
 - wordcloud: for generating visual word frequency clouds
 - csv: for exporting similarity data

2.2 Data Collection and Loading

Users select a directory containing .txt files. The system then loads the contents of these files and stores them for further processing. Each document is uniquely identified by its filename.

2.3 Preprocessing

Text preprocessing is essential to ensure accurate analysis. The following steps are applied:

- Conversion of all text to lowercase
- Removal of punctuation using string operations
- Tokenization using NLTK's word_tokenize
- Elimination of stopwords (commonly used words like "the", "is", etc.)

2.4 Feature Extraction

Once preprocessing is completed, the clean text data is converted into numerical features using the TF-IDF (Term Frequency-Inverse Document Frequency) vectorizer from Scikit-learn. This transformation enables mathematical comparison of the documents based on the frequency and uniqueness of words.

2.5 Similarity Computation

The pairwise similarity between all documents is calculated using cosine similarity on the TF-IDF vectors. The similarity values range from 0 (completely dissimilar) to 1 (identical), and are stored in a matrix format.

2.6 Sentiment Analysis

The VADER (Valence Aware Dictionary and sEntiment Reasoner) model from NLTK is used to assign a sentiment score to each document. This score includes metrics for positivity, negativity, neutrality, and a compound overall sentiment.

2.7 Visualization and Reporting

The system displays results through multiple visual formats:

- **Heatmap:** Shows the similarity matrix with color-coded intensities.

- **Bar Chart:** Displays top 10 most frequent words in each document.
- **Word Cloud:** Visually emphasizes the most common words.

2.8 Plagiarism Detection

A plagiarism threshold (default: 0.8) is used to flag document pairs with suspiciously high similarity scores. Users are notified of these cases in the GUI.

2.9 Export Feature

A complete similarity matrix is exported to a .csv file. This enables further manual review, offline reporting, or integration into other systems.

2.10 Justification of Methodology

This methodology ensures:

- A balanced combination of technical accuracy and user accessibility
- Modular and maintainable design
- Fast execution suitable for multiple files
- Expandability for future enhancements like PDF/DOCX support or cloud integration

3. Implementation

This section focuses on the actual coding and development work involved in building the plagiarism checker system.

The application was developed in Python using the Tkinter library for building the GUI. Each functional feature such as file selection, preprocessing, word cloud generation, similarity matrix computation, and sentiment analysis is implemented as a button-triggered event in the interface. The GUI layout was organized using labels and buttons, ensuring user-friendly interaction and logical workflow.

Text preprocessing is done using the Natural Language Toolkit (NLTK), which includes tokenization, conversion to lowercase, removal of stopwords, and punctuation stripping. These steps help clean and standardize the input data before analysis. The preprocessed text is then transformed into numerical vectors using the TF-IDF (Term Frequency-Inverse Document Frequency) technique from the Scikit-learn library.

Cosine similarity is calculated on the TF-IDF vectors to compare documents. This yields a similarity matrix, which is visualized using Matplotlib as a heatmap to make interpretation intuitive. Word frequency data is also extracted from the cleaned documents and represented through bar charts.

To enhance interpretability, word clouds are generated using the WordCloud library. This visually emphasizes commonly used words in each document. Sentiment analysis is performed using the VADER model from NLTK's SentimentIntensityAnalyzer class, providing polarity scores for each document.

The system also includes functionality to export the similarity matrix as a CSV file, enabling offline access and reporting. A key feature is the plagiarism checker, which identifies document pairs with high similarity (above a predefined threshold) and notifies the user through message boxes.

All code was tested for edge cases like empty files, unsupported file formats, and unreadable files to ensure robust performance. The GUI structure ensures all features are accessible without technical knowledge, making the application usable for non-programmers.

[User] → [GUI App] → [Document Loader] → [Preprocessor] → [TF-IDF Engine] → [Similarity Checker & Visualizer]

4. Result and Analysis

This section showcases the outputs generated by the plagiarism detection system and provides insight into the system's effectiveness through visualizations and quantitative analysis.

Similarity Matrix Output: The system computes cosine similarity scores between all pairs of documents based on their TF-IDF vectors. These scores are presented in a matrix format, highlighting document similarity relationships.

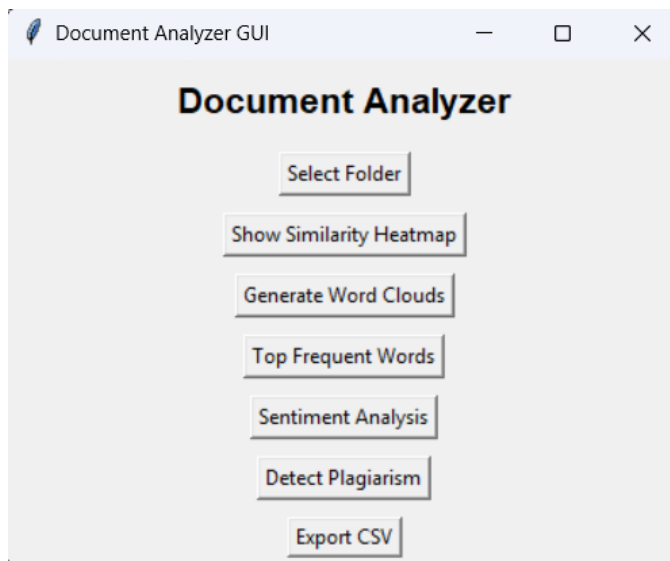
Visual Analysis:

The following graphical outputs demonstrate the key insights derived from the documents:

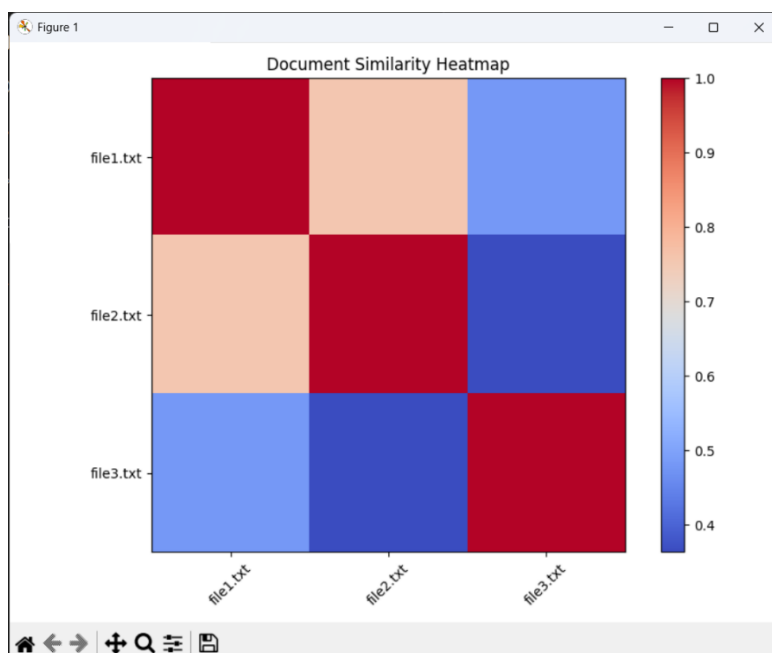
- **Word Clouds:** These highlight the most frequent and significant words in each document, visually emphasizing their thematic content.
- **Bar Charts:** A bar chart representation of the top frequent words per document, useful for identifying repetitive patterns or keyword density.
- **Sentiment Analysis:** Each document is assigned polarity scores, offering a basic understanding of the document's tone (positive, negative, or neutral).
- **Similarity Heatmap:** A color-coded matrix helps visualize document similarities at a glance, aiding in intuitive understanding of overlap.

System Interface and Screenshots: Below are screenshots of the GUI output generated during testing, including file selection, result display, and visualizations:

GUI OUTPUT



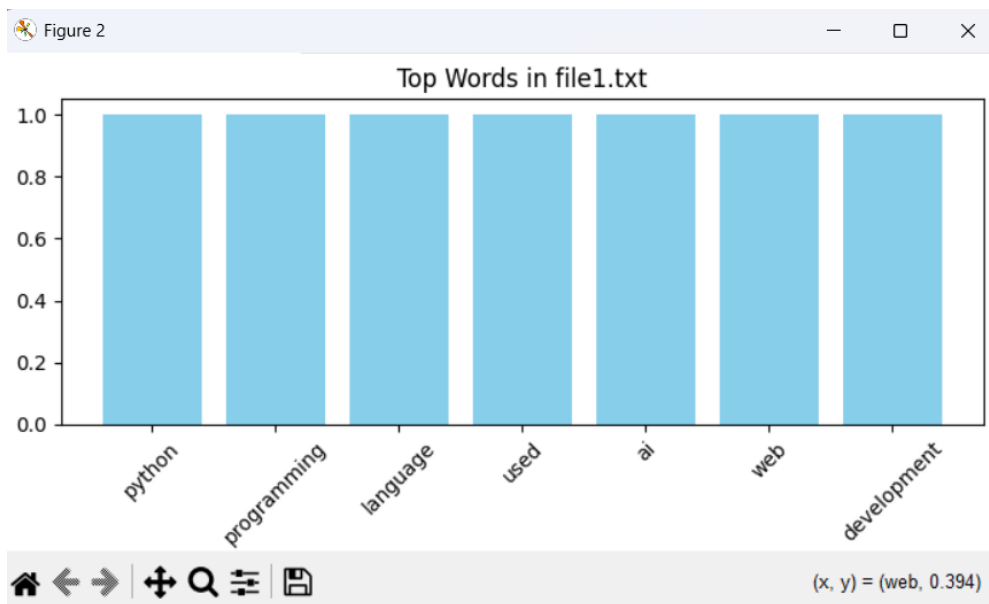
HEATMAP



WORD CLOUD



TOP FREQUENT WORDS



SENTIMENT ANALYSIS

 Sentiment Analysis ×



Sentiment Scores:
file1.txt: {'neg': 0.0, 'neu': 1.0, 'pos': 0.0, 'compound': 0.0}
file2.txt: {'neg': 0.0, 'neu': 0.823, 'pos': 0.177, 'compound': 0.4215}
file3.txt: {'neg': 0.0, 'neu': 0.839, 'pos': 0.161, 'compound': 0.3612}

OK

PLAGIARISM DETECTED

 Plagiarism Detected ×



Potential plagiarism: file1.txt <-> file2.txt (1.00)

OK

5.DISCUSSION AND ANALYSIS

The project successfully meets its intended objectives by offering a user-friendly GUI-based plagiarism detection system. By integrating TF-IDF and cosine similarity measures, the tool effectively identifies document similarities with notable accuracy. The inclusion of features such as word cloud visualization, sentiment analysis, and CSV export enhances the interpretability and practicality of the system.

One of the key strengths of the tool lies in its intuitive interface, which makes it accessible even to users with limited technical expertise. Additionally, the use of established Python libraries ensures reliable performance across various test cases.

However, the system does have certain limitations. Currently, it only supports .txt file formats, which restricts its utility in more diverse document environments. Moreover, the analysis is based solely on surface-level textual similarity and does not account for paraphrasing or semantic equivalence.

Despite these limitations, the tool provides a valuable solution for detecting textual plagiarism and maintaining academic integrity. With its core functionalities working as intended, it offers a solid foundation for further enhancement and extension.

6. Future Scope

While the current plagiarism detection system effectively identifies textual similarities and provides visual analysis, there are several directions in which the project can be enhanced:

- **Integration of Deep Learning Models:** Incorporating transformer-based models like BERT or RoBERTa can improve semantic understanding and detect paraphrased or contextually similar content more accurately.
- **Multilingual Support:** Extending the system to analyze documents written in different languages using language detection and translation pipelines.
- **Real-time Web Deployment:** Hosting the application as a web-based service with live document analysis and cloud-based storage for collaborative environments.
- **Source Code Plagiarism Detection:** Enhancing the tool to support code-based plagiarism detection through abstract syntax tree (AST) analysis or token-based similarity techniques.
- **Database Connectivity:** Linking the system to a central repository for checking against previously submitted documents or institutional databases.
- **Enhanced User Feedback:** Providing detailed plagiarism reports with highlighted matched segments and suggestions for improvement.

7.References

1. Mueller, A. (2015). Wordcloud. *GitHub Repository*.
https://github.com/amueller/word_cloud
2. Python Software Foundation. (2024). *Python Language Reference, version 3.10*.
<https://www.python.org/>
3. Tkinter Docs. (2024). *Tkinter GUI Programming*.
<https://docs.python.org/3/library/tkinter.html>